Lehrerexemplar · Boss-Kampf 5 · Welt 5 · DS 36

Erwartungshorizont: Die drei Baumhüter

Erwartungshorizont / Bewertungsbogen für _____ Stationen: ☐ A ☐ B ☐ C

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten · gewertet werden zwei Stationen (je 12 BE, zusammen 24 BE)

Erwartete Prüfungsleistungen	I	II	III	BE
Station A – Die Glutwächterin (Max-Heap)				
A a) Max-Heap-Eigenschaften prüfen Kernpunkte: H1 ist ein Max-Heap; H2 verletzt die Form-Eigenschaft (Lücke in der untersten Ebene); H3 verletzt die Eltern-Max-Eigenschaft (61 > 60).	2			
A b) Einfügen und Entfernen des größten Elements von Hand Kernpunkte: 77 wird linkes Kind von 58 und steigt über 58 und 64 auf (77 < 90); Entfernen: 90 geliefert, 58 (letzter Knoten) in die Wurzel, versickert über 77 und 64.	1	2		
A c) Max-Heap und binären Suchbaum vergleichen Kernpunkte: Größtes Element: Heap in der Wurzel (ein Zugriff), BST ganz rechts (Weg bis zur Höhe, bei Entartung n); Suche: BST nur ein Teilbaum je Schritt (höchstens Höhe), Heap keine Links-rechts-Ordnung → im schlimmsten Fall alle Knoten; Heap durch Form-Eigenschaft stets ausgeglichen.		1	1	
A d) Rekursives Struktogramm isMaxHeap; Begründung Kernpunkte: Kopf mit Signatur; Basisfall leerer Baum → wahr; Vergleich mit linkem und rechtem Kind jeweils nur nach Leer-Abfrage; rekursive Aufrufe für beide Teilbäume, mit und verknüpft; Rückgabe in jedem Pfad; Notation nach EH §5; Begründung: ≥ ist transitiv – gilt Eltern ≥ Kind für jeden Knoten, so ist jeder Knoten ≥ allen Nachfahren; beim BST hängt die Bedingung von allen Vorfahren und der Seite ab.	1	3	1	
Summe Station A	4	6	2	**/ 12**
Station B – Der Runenschreiber (Huffman-Baum)				
B a) Huffman-Baum nach Verfahren erstellen; Codewörter Kernpunkte: Verbindungen D+L (2), T+A (3), E+DL (4), TA+N (7), EDL+TAN (11) mit Einordnung hinter gleich häufige; Codewörter D = 010, L = 011, T = 100, A = 101, E = 00, N = 11; 27 Bit (statt 33 Bit bei 3 Bit je Zeichen).	1	3		
B b) Bitfolge decodieren Kernpunkte: Zerlegung 110 | 01 | 00 | 10 | 00 | 111 → REISIG.	2			
B c) Präfixfreiheit begründen; Vorschlag beurteilen Kernpunkte: Zeichen stehen nur in Blättern, ein Codewort ist der Weg bis zu einem Blatt; wäre ein Codewort Anfang eines anderen, läge auf dem Weg zum zweiten Blatt ein Blatt – Blätter haben aber keine Kinder; Vorschlag: I = 0 ist Anfang von E = 01, nicht präfixfrei, z. B. 0110 = E S oder I R → nicht eindeutig decodierbar, abzulehnen.		1	1	
B d) Implementierung: decodiere auf BinTree Kernpunkte: Methodenkopf public static String decodiere(BinTree baum, String bits); Hilfsverweis t beginnt bei baum; Durchlauf aller Bits mit charAt und length; '0' → getLeft, sonst getRight; ist t ein Blatt: Zeichen mit Cast (String) anhängen und t wieder auf baum setzen; Rückgabe des Texts (leere Bitfolge → ""); Codebaum bleibt unverändert.	1	2	1	
Summe Station B	4	6	2	**/ 12**
Station C – Die Archivarin (B-Baum)				

Erwartete Prüfungsleistungen	I	II	III	BE
C a) Ordnungsprinzip eines B-Baums beschreiben Kernpunkte: Schlüssel in jedem Knoten aufsteigend sortiert; ein innerer Knoten mit k Schlüsseln teilt den Wertebereich in k + 1 Bereiche: links vom ersten Schlüssel kleinere, zwischen zwei Schlüsseln die dazwischenliegenden, rechts vom letzten größere (Verallgemeinerung des BST); alle Blätter auf einer Ebene.	2	1		
C b) Suche im B-Baum Kernpunkte: 42: [30 60] → mittleres Kind [35 42 50], gefunden; 65: [30 60] → rechtes Kind [70 85], 65 < 70 im Blatt → nicht enthalten.	1	1		
C c) Einfügen mit Teilung (Split) angeben Kernpunkte: 25 → [10 20 25] ohne Teilung; 45 → Überlauf [35 42 45 50], 45 steigt auf, Wurzel [30 45 60] mit Kindern [35 42], [50]; 15 → Überlauf [10 15 20 25], 20 steigt auf, Wurzel [20 30 45 60] läuft über → 45 neue Wurzel, Kinder [20 30] und [60], Höhe 3.	1	3		
C d) Suchaufwand abschätzen und beurteilen Kernpunkte: Ordnung 100: höchstens 99 Schlüssel je Knoten; 3 Ebenen fassen höchstens 999 999 Schlüssel → mindestens 3 Ebenen; bei Mindestbelegung (49 Schlüssel, 50 Kinder) fassen 4 Ebenen mindestens 249 999, 5 Ebenen mindestens 12 499 999 → höchstens 4 Ebenen, also höchstens 4 Knoten; ausgeglichener BST: 19 Ebenen; vorteilhaft, wenn ein Knotenzugriff teuer ist (Festplatte/Datenbank: ein Knoten = ein Blockzugriff), Vergleiche im Arbeitsspeicher sind billig.		1	2	
Summe Station C	4	6	2	**/ 12**
Gesamt (zwei gewählte Stationen)				**/ 24**

AFB-Verteilung je Station: I 4 BE (33 %) · II 6 BE (50 %) · III 2 BE (17 %) – bei jeder Wahl von zwei Stationen also I 8 · II 12 · III 4 von 24 BE (Abitur-Richtwert 30 / 53 / 17 %). Zeitbudget: 24 BE · 2,5 min = 60 min.

Die gewählten Lösungsansätze und -wege müssen nicht mit den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (EH 2025) entsprechend zu bewerten. Laufzeitfehler, die nicht durch explizite Abfragen ausgeschlossen werden, führen zu Abzug (EH §4). Bearbeitet ein Prüfling alle drei Stationen, werden die zwei zuerst angekreuzten gewertet.

Auswertung des Boss-Kampfs (Abi-Training, fließt nicht in die Note ein)

- Papier-Boss: Auswertung nach Kernpunkten durch die Lehrkraft oder als Selbst-/Peer-Korrektur mit diesem Erwartungshorizont; Ergebnis in BE auf dem Antwortbogen.
- Papier-Compiler zu B d) (bzw. A d) als zeilengetreue Java-Übertragung des Struktogramms): alle 8 Tests der passenden Testklasse grün; auf dem Antwortbogen ankreuzen, ob der **erste Versuch** grün war.
- Falle P1 im Blick: kein Zugriff auf einen leeren Baum (A d): Leer-Abfrage vor jedem Kind-Zugriff; B d): Rücksprung zur Wurzel, Tests leereBitfolge und baumUnveraendert grün).
- Zusätzlich im Lab: **Online-Boss „Die drei Baumhüter – Online-Duell“** (25 min, automatische Auswertung, bester Versuch zählt; Punkte und Abzeichen „… besiegt“ vergibt das Lab automatisch, Ergebnisse in der Lehrkraft-Auswertung).
- Orientierung (informativ, Abi-Skala): ab 95 % 15 P · 85 % 13 P · 75 % 11 P · 65 % 9 P · 55 % 7 P · 45 % 5 P · 40 % 4 P · 27 % 2 P. Für 24 BE: 23 BE ≈ 15 P, 18 BE ≈ 11 P, 10 BE ≈ 4 P.

Musterlösungen, Kernpunkte und typische Fehler

Station A Die Glutwächterin – Max-Heap

A a) Max-Heap-Eigenschaften prüfen 2 BE (I 2 · II 0 · III 0)

Musterlösung

H1: Max-Heap. Ebenen 0 und 1 sind voll, in der untersten Ebene stehen 10, 35, 20 lückenlos von links; jeder Knoten ist mindestens so groß wie seine Kinder (50 ≥ 40, 45; 40 ≥ 10, 35; 45 ≥ 20).

H2: kein Max-Heap – die **Form-Eigenschaft** ist verletzt: In der untersten Ebene fehlt das rechte Kind von 40, obwohl rechts davon (Kind von 45) ein Knoten steht (Lücke). Die Eltern-Max-Eigenschaft ist erfüllt.

H3: kein Max-Heap – die **Eltern-Max-Eigenschaft** ist verletzt: 61 ist größer als sein Elternknoten 60. Die Form-Eigenschaft ist erfüllt.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
H1 als Max-Heap erkannt; H2: Form-Eigenschaft verletzt (Lücke)	I	1	☐
H3: Eltern-Max-Eigenschaft verletzt (61 > 60)	I	1	☐

Alternativen / Teilpunkte: Eine Begründung über die Reihung in Ebenenreihenfolge (H2 hat an Position 4 eine Lücke) ist gleichwertig.

Typische Fehler

- H2 als Max-Heap eingestuft, weil alle Werte passen (Form-Eigenschaft übersehen).
- H3 wegen 75 > 70 „falsch herum“ für fehlerhaft gehalten – entscheidend ist Eltern ≥ Kind.
- Max-Heap mit BST verwechselt (links kleiner, rechts größer gefordert).

A b) Einfügen und Entfernen des größten Elements von Hand 3 BE (I 1 · II 2 · III 0)

Musterlösung

Einfügen von 77: Die nächste freie Stelle ist das linke Kind von 58. Tausch mit 58 (77 > 58), dann mit 64 (77 > 64), dann Stopp (77 < 90). Ergebnis in Ebenenreihenfolge: 90, 77, 72, 31, 64, 70, 15, 12, 27, 58 (grau: veränderte Knoten).

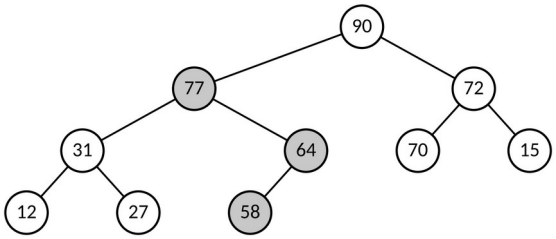


Abbildung EH.A1: Max-Heap nach dem Einfügen von 77

Entfernen des größten Elements: 90 wird geliefert. Der letzte Knoten (58, unterste Ebene ganz rechts) ersetzt die Wurzel und wird entfernt. 58 < 77 und 58 < 72 → Tausch mit dem größeren Kind 77; dann 58 < 64 → Tausch mit 64; 58 ist jetzt ein Blatt. Ergebnis: 77, 64, 72, 31, 58, 70, 15, 12, 27 – wieder ein Max-Heap (geprüft).

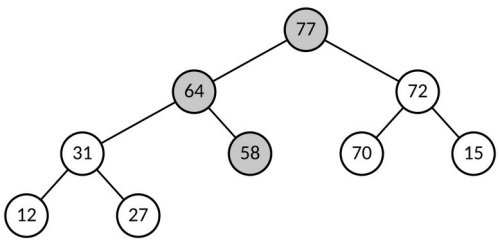


Abbildung EH.A2: Max-Heap nach dem Entfernen des größten Elements

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Einfügeposition: nächste freie Stelle (linkes Kind von 58)	I	1	☐
Aufsteigen von 77 bis unter 90	II	1	☐
Entfernen: letzter Knoten in die Wurzel, Versickern mit dem größeren Kind (77, dann 64)	II	1	☐

Alternativen / Teilpunkte: Folgefehler aus dem Einfügen werden beim Entfernen nicht erneut abgezogen, wenn das Verfahren richtig angewendet wird. Die Darstellung als Reihung in Ebenenreihenfolge ist gleichwertig.

Typische Fehler

- 77 als rechtes Kind von 31 oder unter 72 eingefügt (Form-Eigenschaft nicht beachtet).
- Beim Versickern mit dem kleineren Kind getauscht (72 statt 77) → Eltern-Max verletzt.
- Statt des letzten Knotens das größere Kind in die Wurzel gezogen (Lücke entsteht).

A c) Max-Heap und binären Suchbaum vergleichen 2 BE (I 0 · II 1 · III 1)

Musterlösung

Größtes Element: Im Max-Heap steht es immer in der Wurzel – ein einziger Zugriff, unabhängig von der Anzahl der Knoten. Im binären Suchbaum steht es im Knoten ganz rechts; man muss von der Wurzel aus immer nach rechts gehen (so viele Schritte wie die Höhe, bei einem entarteten Baum bis zu n).

Suche nach einem beliebigen Wert: Im BST entscheidet jeder Vergleich, ob links oder rechts weitergesucht wird – höchstens so viele Vergleiche wie die Höhe. Der Max-Heap kennt keine Ordnung zwischen linkem und rechtem Teilbaum; ein gesuchter Wert kann in beiden stehen. Man kann nur Teilbäume überspringen, deren Wurzel kleiner als der gesuchte Wert ist, muss aber im schlimmsten Fall alle Knoten untersuchen. Dafür ist der Heap wegen der Form-Eigenschaft immer ausgeglichen (Höhe ≈ $\log_2 n$), ein BST kann entarten. Für die Glutwächterin, die immer den heißesten Stein braucht, ist der Max-Heap die bessere Wahl.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Zugriff auf das größte Element: Wurzel (Heap) vs. Weg nach ganz rechts (BST)	II	1	☐
Suche: ein Teilbaum je Schritt (BST) vs. ggf. alle Knoten (Heap), begründet über die jeweilige Ordnung	III	1	☐

Alternativen / Teilpunkte: Auch weitere sachgerechte Kriterien (Einfügen, Ausgeglichenheit, sortierte Ausgabe per Inorder beim BST) können gewürdigt werden, ersetzen aber nicht die beiden geforderten Aspekte.

Typische Fehler

- Behauptung, im Heap könne man wie im BST binär suchen.
- Nur Unterschiede aufgezählt, ohne sie auf Zugriff und Suche zu beziehen.

A d) Rekursives Struktogramm isMaxHeap; Begründung 5 BE (I 1 · II 3 · III 1)

Musterlösung

Musterlösung (zeilengetreu in Java übertragen und mit den Papier-Compiler-Tests geprüft: HeapTest 8/8 grün):

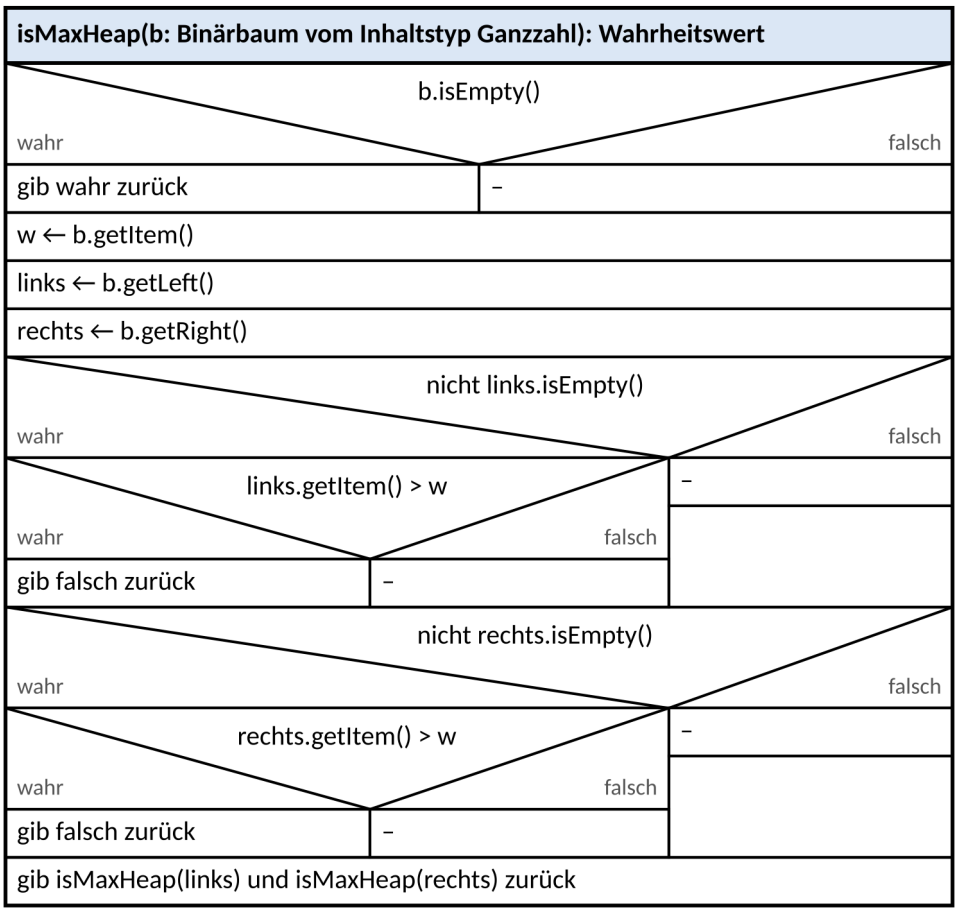


Abbildung EH.A3: Struktogramm der Operation isMaxHeap

Begründung: Die Relation „größer oder gleich“ ist transitiv. Ist jeder Knoten mindestens so groß wie seine Kinder, dann ist er auch mindestens so groß wie deren Kinder usw., also wie **alle** Knoten in seinen Teilbäumen. Es genügt daher, jede Eltern-Kind-Kante einmal zu prüfen. Beim binären Suchbaum reicht das nicht: Ein Knoten im rechten Teilbaum des linken Kindes ist größer als dieses Kind, muss aber trotzdem kleiner als die Wurzel sein – diese Bedingung folgt nicht aus den Eltern-Kind-Vergleichen, deshalb braucht isBST Grenzen (min, max) aus allen Vorfahren.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Kopf mit Signatur; Basisfall leerer Baum → wahr	I	1	☐
Vergleich mit beiden Kindern, jeweils erst nach der Leer-Abfrage	II	1	☐
Rekursion für beide Teilbäume, Verknüpfung mit und, Rückgabe in jedem Pfad	II	1	☐
Notation nach EH §5 (←, gib … zurück, Verzweigungen)	II	1	☐
Begründung über die Transitivität, Abgrenzung zum BST	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (ebenfalls 8/8 Tests grün, siehe Anhang): Basisfall Blatt mit Leer-Abfragen vor jedem Kind-Zugriff; Hilfsoperation mit Obergrenze (Wert des Elternknotens), Start mit dem Wert der Wurzel (b ist nicht leer). Eine Bedingung „nicht links.isEmpty() und links.getItem() > w“ ist zulässig, wenn auf die Auswertung von links nach rechts (Kurzschluss) hingewiesen wird; ohne Hinweis 1 BE Abzug höchstens einmal. Java-Code statt Struktogramm: höchstens 3 von 4 BE für das Struktogramm.

Typische Fehler

- getItem() auf einem leeren Kind (Knoten mit nur einem Kind) – Laufzeitfehler.
- Nur Wurzel und Kinder geprüft, keine Rekursion.
- Rekursion nur in den linken Teilbaum oder Verknüpfung mit oder.
- ≥ statt > (gleiche Werte als Verstoß gewertet).
- Begründung „weil es ein Heap ist“ ohne Transitivität.

Station B Der Runenschreiber – Huffman-Baum

B a) Huffman-Baum nach Verfahren erstellen; Codewörter 4 BE (I 1 · II 3 · III 0)

Musterlösung

Ablauf des Verfahrens (Liste nach jedem Schritt, Häufigkeit hinter dem Doppelpunkt; mit einer Python-Fassung des Verfahrens erzeugt und geprüft):

Schritt	Liste der Bäume (vorn zuerst; Zeichen des Baums : Häufigkeit)
Start	D:1 | L:1 | T:1 | A:2 | E:2 | N:4
1	T:1 | A:2 | E:2 | DL:2 | N:4
2	E:2 | DL:2 | TA:3 | N:4
3	TA:3 | N:4 | EDL:4
4	EDL:4 | TAN:7
5	EDLTAN:11

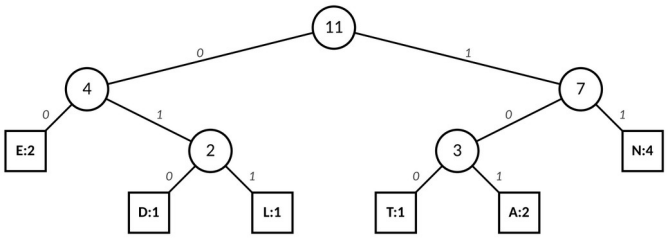


Abbildung EH.B1: Huffman-Baum für TANNENNADEL (innere Knoten: Summe der Häufigkeiten)

Codewörter: D = 010, L = 011, T = 100, A = 101, E = 00, N = 11. TANNENNADEL = 100 101 11 11 00 11 11 101 010 00 011 → **27 Bit** (zum Vergleich: 33 Bit bei einem festen 3-Bit-Code). Probe: Die Java-Musterlösung zu d) decodiert diese Bitfolge mit dem Baum wieder zu TANNENNADEL.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Erste Schritte (D+L, T+A) und Einordnung hinter gleich häufige Bäume	II	1	☐
vollständiger Huffman-Baum	II	1	☐
Codewörter abgelesen (links 0, rechts 1)	I	1	☐
Anzahl der Bits: 27 (gewichtete Summe der Codelängen)	II	1	☐

Alternativen / Teilpunkte: Wird die Einordnungsregel nicht genau eingehalten, entsteht ggf. ein anderer, ebenfalls optimaler Baum (auch dann 27 Bit): höchstens 1 BE Abzug. Folgefehler bei Codewörtern und Bitzahl werden nicht erneut abgezogen.

Typische Fehler

- Neuer Baum vor statt hinter gleich häufige Bäume eingeordnet.
- Links/rechts beim Verbinden vertauscht.
- Bitzahl als Summe der Codelängen ohne Häufigkeiten (16 statt 27).

B b) Bitfolge decodieren 2 BE (I 2 · II 0 · III 0)

Musterlösung

Von der Wurzel aus je Bit nach links (0) bzw. rechts (1), bei einem Blatt das Zeichen notieren und wieder bei der Wurzel beginnen: 110 → R, 01 → E, 00 → I, 10 → S, 00 → I, 111 → G. Ergebnis: **REISIG** (Reisig – trockene Zweige).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Zerlegung in Codewörter	I	1	☐
Ergebnis REISIG	I	1	☐

Alternativen / Teilpunkte: Die Angabe des Ergebnisses mit erkennbarer Zerlegung genügt.

Typische Fehler

- Nach einem Blatt nicht wieder an der Wurzel begonnen.
- 0 und 1 vertauscht.

B c) Präfixfreiheit begründen; Vorschlag beurteilen 2 BE (I 0 · II 1 · III 1)

Musterlösung

Begründung: Im Codebaum stehen Zeichen nur in Blättern, das Codewort eines Zeichens ist der Weg von der Wurzel zu seinem Blatt. Wäre ein Codewort u der Anfang eines anderen Codeworts v, so müsste der Weg zu v über das Blatt von u hinausführen – ein Blatt hat aber keine Kindknoten. Also ist kein Codewort Anfang eines anderen: Der Code ist präfixfrei. Deshalb weiß man beim Decodieren an jedem Blatt sicher, dass ein Zeichen zu Ende ist – Trennzeichen sind nicht nötig.

Beurteilung: Der Vorschlag ist abzulehnen. I = 0 ist der Anfang von E = 01, der Code ist nicht präfixfrei. Die Bitfolge 0110 lässt sich als 01|10 = ES oder als 0|110 = IR lesen – ohne Trennzeichen ist sie nicht eindeutig decodierbar. Im Codebaum müsste I in einem inneren Knoten stehen; ein solcher Baum ist kein gültiger Codebaum.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Begründung über Blätter und Wege im Codebaum	II	1	☐
Beurteilung mit Gegenbeispiel (Mehrdeutigkeit)	III	1	☐

Alternativen / Teilpunkte: Jedes korrekte Gegenbeispiel ist gleichwertig (z. B. auch 010 = EI oder IS). Wird nur festgestellt, dass I der Anfang von E ist, ohne die Folge (Mehrdeutigkeit beim Decodieren ohne Trennzeichen) zu erläutern, ist der Beurteilungspunkt nicht erreicht.

Typische Fehler

- Präfixfreiheit nur definiert, nicht begründet.
- Vorschlag gelobt, weil I kürzer ist (Kürze ≠ Decodierbarkeit).

B d) Implementierung: decodiere auf BinTree 4 BE (I 1 · II 2 · III 1)

Musterlösung

Musterlösung (Java, mit javac und den Papier-Compiler-Tests geprüft: HuffmanTest 8/8 grün; Beispiel REISIG, leere Bitfolge, Baum aus a) → TANNENNADEL, Baum danach unverändert):

```
public static String decodiere(BinTree baum, String bits) {
    String text = "";
    BinTree t = baum;
    for (int i = 0; i < bits.length(); i++) {
        if (bits.charAt(i) == '0') {
            t = t.getLeft();
        } else {
            t = t.getRight();
        }
        if (t.isLeaf()) {
            text = text + (String) t.getItem();
            t = baum;
        }
    }
    return text;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf, Ergebnisvariable, Rückgabe	I	1	☐
Durchlauf aller Bits; '0' → links, '1' → rechts (Zeichenvergleich)	II	1	☐
Blatt erkennen (isLeaf), Zeichen mit Cast (String) anhängen	II	1	☐
Rücksprung zur Wurzel nach jedem Blatt, Codebaum unverändert, leere Bitfolge → ""	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (ebenfalls 8/8 Tests grün): rekursive Fassung mit dem Index des nächsten Bits; zwei geschachtelte Schleifen (außen je Zeichen, innen bis zum Blatt), auch mit substring(i, i + 1).equals("0"). Der Vergleich bits.charAt(i) == '0' vergleicht Zeichen und ist richtig; == 0 (Zahl) ist falsch. Fehlende Casts: 1 BE Abzug höchstens einmal (im Papier-Compiler fällt `text + t.getItem()` nicht auf, ist aber im Abitur mit Cast zu schreiben).

Alternative (1): rekursiv

```
public static String decodiere(BinTree baum, String bits) {
    return dec(baum, baum, bits, 0);
}

// rekursiv: t = aktueller Knoten, i = naechstes Bit
public static String dec(BinTree baum, BinTree t, String bits, int i) {
    if (t.isLeaf()) {
        return (String) t.getItem() + dec(baum, baum, bits, i);
    }
    if (i == bits.length()) {
        return "";
    }
    if (bits.charAt(i) == '0') {
        return dec(baum, t.getLeft(), bits, i + 1);
    }
    return dec(baum, t.getRight(), bits, i + 1);
}
```

Alternative (2): geschachtelte Schleifen

```
public static String decodiere(BinTree baum, String bits) {
    String text = "";
    int i = 0;
    while (i < bits.length()) {
        BinTree t = baum;
        while (!t.isLeaf()) {
            if (bits.substring(i, i + 1).equals("0")) {
                t = t.getLeft();
            } else {
                t = t.getRight();
            }
            i++;
        }
        text = text + (String) t.getItem();
    }
    return text;
}
```

Typische Fehler

- Nach einem Blatt nicht zur Wurzel zurückgekehrt (Tests rot).
- baum selbst weiterbewegt (baum = baum.getLeft()) – der Weg zurück zur Wurzel ist verloren.
- Blatt-Prüfung vor dem Schritt → das letzte Zeichen fehlt.
- Vergleich mit der Zahl 0 statt mit dem Zeichen '0'.
- Schleife bis bits.length() – 1 (letztes Bit fehlt).

Station C Die Archivarin – B-Baum

C a) Ordnungsprinzip eines B-Baums beschreiben 3 BE (I 2 · II 1 · III 0)

Musterlösung

In jedem Knoten sind die Item-Nummern aufsteigend sortiert. Ein innerer Knoten mit k Nummern hat k + 1 Kindknoten: Im Kind links von der ersten Nummer liegen nur kleinere Nummern (10, 20 < 30), im Kind zwischen zwei benachbarten Nummern nur Nummern dazwischen (35, 42, 50 liegen zwischen 30 und 60), im Kind rechts von der letzten Nummer nur größere (70, 85 > 60). Das gilt in jedem Teilbaum. Der B-Baum verallgemeinert damit den binären Suchbaum (dort: ein Schlüssel, zwei Bereiche). Alle Blätter liegen auf derselben Ebene.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Schlüssel im Knoten aufsteigend sortiert	I	1	☐
Kinder links / zwischen / rechts mit Bezug auf Abbildung C.2	I	1	☐
Verallgemeinerung des BST: k Schlüssel → k + 1 Bereiche (für jeden Teilbaum)	II	1	☐

Alternativen / Teilpunkte: Eine Beschreibung ausschließlich am Beispiel ist zulässig, wenn das Prinzip erkennbar verallgemeinert wird.

Typische Fehler

- Nur „sortiert“ angegeben, ohne die Lage der Kindknoten.
- Knoten als Binärbaum mit drei Kindern gedeutet.

C b) Suche im B-Baum 2 BE (I 1 · II 1 · III 0)

Musterlösung

42: Wurzel [30 60]: 30 < 42 < 60 → mittleres Kind [35 42 50] → gefunden (2 Knoten).

65: Wurzel [30 60]: 65 > 60 → rechtes Kind [70 85]: 65 < 70 → 65 müsste links von 70 stehen, das Blatt hat aber keine Kindknoten. Damit ist 65 nicht enthalten (2 Knoten) – mehr Knoten kommen für 65 nach dem Ordnungsprinzip nicht in Frage.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Suchwege für 42 und 65	I	1	☐
Begründung: Suche endet im Blatt ohne Treffer	II	1	☐

Alternativen / Teilpunkte: Die Angabe der Vergleiche innerhalb der Knoten ist nicht verlangt.

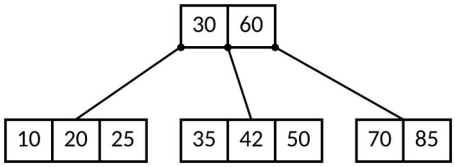
Typische Fehler

- Für 65 auch das mittlere Kind untersucht.
- Suche nach 65 bei 60 beendet.

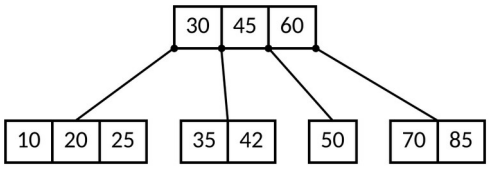
C c) Einfügen mit Teilung (Split) angeben 4 BE (I 1 · II 3 · III 0)

Musterlösung

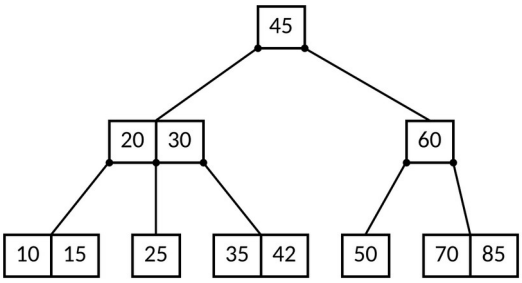
Ergebnisse (mit einer Python-Fassung des Verfahrens aus Abbildung C.3 erzeugt und geprüft):



nach dem Einfügen von 25: Platz im Blatt, keine Teilung



nach dem Einfügen von 45: Blatt [35 42 45 50] läuft über, der dritte Schlüssel 45 steigt auf



nach dem Einfügen von 15: Blatt [10 15 20 25] läuft über, 20 steigt auf; die Wurzel [20 30 45 60] läuft über und wird geteilt, 45 wird neue Wurzel – der Baum wächst um eine Ebene

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
25 ohne Teilung	I	1	☐
45: Teilung des Blatts, 45 in der Wurzel	II	1	☐
15: Teilung des Blatts, 20 steigt auf	II	1	☐
Teilung der Wurzel, neue Wurzel 45, Kindknoten richtig verteilt	II	1	☐

Alternativen / Teilpunkte: Folgefehler werden nicht erneut abgezogen. Wird bei der Teilung der zweite statt des dritten Schlüssels hochgereicht (andere übliche Variante), ist dies bei konsequenter Anwendung mit 1 BE Abzug zu bewerten, da das Material die Position vorgibt.

Typische Fehler

- Kindknoten bei der Teilung der Wurzel nicht mit aufgeteilt.
- Überlauf schon bei 3 Schlüsseln angenommen.
- Neuen Schlüssel in einen inneren Knoten statt ins Blatt eingefügt.

C d) Suchaufwand abschätzen und beurteilen 3 BE (I 0 · II 1 · III 2)

Musterlösung

Abschätzung: Ein Knoten enthält höchstens 99 Schlüssel und hat höchstens 100 Kinder. Mit 3 Ebenen passen höchstens $99 + 100 \cdot 99 + 100^2 \cdot 99 = 999\,999$ Schlüssel in den Baum – 500 000 Nummern benötigen also mindestens 3 Ebenen. Bei der Mindestbelegung (Wurzel 1 Schlüssel, sonst 49 Schlüssel und 50 Kinder) enthalten 4 Ebenen mindestens 249 999, 5 Ebenen aber mindestens 12 499 999 Schlüssel. Der Baum hat daher 3 oder 4 Ebenen: Eine Suche untersucht **höchstens 4 Knoten**.

Vergleich: Ein ausgeglichener binärer Suchbaum mit 500 000 Schlüsseln hat $\lceil \log_2(500\,001) \rceil = 19$ Ebenen – bis zu 19 Knoten werden untersucht. Innerhalb eines B-Baum-Knotens sind dafür bis zu 99 Schlüssel zu durchsuchen (mit binärer Suche höchstens 7 Vergleiche), insgesamt also eine ähnliche Zahl von Vergleichen.

Beurteilung: Der B-Baum ist vorteilhaft, wenn das Laden eines Knotens viel teurer ist als ein Vergleich – z. B. wenn der Baum auf einer Festplatte oder in einer Datenbank liegt und jeder Knoten als ein Block gelesen wird (4 statt 19 Blockzugriffe). Liegt alles im Arbeitsspeicher, bringt er kaum Vorteile.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Abschätzung der Ebenenzahl (mindestens 3 über die Maximalbelegung bzw. Logarithmus zur Basis 100)	II	1	☐
Vergleich mit dem ausgeglichenen BST (≈ 19 Ebenen), höchstens 4 Knoten durch Mindestbelegung begründet	III	1	☐
Beurteilung: teure Knotenzugriffe (Festplatte/Datenbank) vs. billige Vergleiche im Knoten	III	1	☐

Alternativen / Teilpunkte: Eine Abschätzung über $\log_{100}(500\,000) \approx 2{,}85 \rightarrow 3$ Ebenen bzw. über $\log_{50}$ für die Mindestbelegung ist gleichwertig. Ohne Mindestbelegung (nur „3 Ebenen“) höchstens 2 von 3 BE.

Typische Fehler

- Schlüssel und Knoten verwechselt (500 000 / 99 Knoten „durchsuchen“).
- Vergleich ohne Zahlen („B-Baum ist flacher“).
- Beurteilung ohne Bezug auf die Kosten eines Knotenzugriffs.

Anhang: Papier-Compiler-Workspace zu B d) und A d)

Für den Papier-Compiler legst du in der Online-IDE einen Workspace „W5 Boss 5 – Papier-Compiler“ mit der Klasse Baumhueter (Gerüst unten: beide Methoden mit leerem Rumpf) und den Testklassen HeapTest und HuffmanTest an, aktivierst die Bibliothek „Niedersachsen“ (Rechtsklick auf den Workspace → Einstellungen → Häkchen) und verteilst ihn über „An Klasse austeilen“. Wer Station B gewählt hat, tippt die Handschrift aus B d) **unverändert** in decodiere ab und startet HuffmanTest. Wer B nicht gewählt hat, überträgt das Struktogramm aus A d) Zeile für Zeile in isMaxHeap und startet HeapTest. Erst danach dürfen Fehler korrigiert werden (dann ohne Erster-Versuch-Bonus).

Vor Einsatz prüfen

Bibliothek „Niedersachsen“ im Workspace aktivieren – ohne sie ist BinTree unbekannt. Die Casts `(Integer) b.getItem()` und `(String) t.getItem()` (Unboxing beim Vergleich mit >) und den JUnit-Dialekt (parameterloser Konstruktor der Testklasse, @Test, globale assertEquals/assertTrue/assertFalse mit Meldung als letztem Argument) vor dem ersten Einsatz mit einem Probedurchlauf testen – insbesondere die Meldung des Testrunners bei einer IllegalStateException (Mutante „Leer-Abfrage der Kinder fehlt“): Der Testrunner muss danach weiterlaufen. Die Hilfsmethoden k, blatt, leer, z und codebaum der Testklassen sind private und ohne @Test. Geprüft wurde mit javac (Java 21) und einem Nachbau von BinTree mit denselben Exceptions.

Klasse Baumhueter (Gerüst für den Workspace)

```
public class Baumhueter {

    // Station A d): Struktogramm zeilengetreu in Java
    public static boolean isMaxHeap(BinTree b) {
        // HIER Station A d) (Struktogramm zeilengetreu in Java) bzw. leer lassen
        return false;
    }

    // Station B d): Handschrift unveraendert abtippen
    public static String decodiere(BinTree baum, String bits) {
        // HIER die Handschrift aus Station B d) UNVERAENDERT abtippen
        return "";
    }
}
```

Testklasse HeapTest (8 Tests zu isMaxHeap)

```
public class HeapTest {

    public HeapTest() {
    }

    // Hilfsmethode: Knoten mit Wert v und den Teilbaeumen l und r
    private BinTree k(int v, BinTree l, BinTree r) {
        BinTree b = new BinTree(v);
        b.setLeft(l);
        b.setRight(r);
        return b;
    }

    private BinTree blatt(int v) {
        return new BinTree(v);
    }

    private BinTree leer() {
        return new BinTree();
```

```
    }

    // Max-Heap aus Abbildung A.1
    private BinTree heapA1() {
        return k(90, k(64, k(31, blatt(12), blatt(27)), blatt(58)), k(72, blatt(70), blatt(15)));
    }

    @Test
    public void abbildungA1() {
        assertTrue(Baumhueter.isMaxHeap(heapA1()), "Abbildung A.1 ist ein Max-Heap");
    }

    @Test
    public void knotenMitEinemKind() {
        BinTree h = k(90, k(77, k(31, blatt(12), blatt(27)), k(64, blatt(58), leer())), k(72,
blatt(70), blatt(15)));
        assertTrue(Baumhueter.isMaxHeap(h), "Heap nach dem Einfuegen von 77 (64 hat nur ein Kind)");
    }

    @Test
    public void verletzungKind() {
        BinTree h = k(80, k(60, blatt(30), blatt(61)), k(75, blatt(70), leer()));
        assertFalse(Baumhueter.isMaxHeap(h), "61 ist groesser als 60");
    }

    @Test
    public void verletzungTief() {
        BinTree h = k(90, k(64, k(31, blatt(12), blatt(40)), blatt(58)), k(72, blatt(70), blatt(15)));
        assertFalse(Baumhueter.isMaxHeap(h), "40 ist groesser als 31 (Ebene 3)");
    }

    @Test
    public void verletzungRechts() {
        BinTree h = k(90, k(64, k(31, blatt(12), blatt(27)), blatt(58)), k(72, blatt(70), blatt(80)));
        assertFalse(Baumhueter.isMaxHeap(h), "80 ist groesser als 72 (rechter Teilbaum)");
    }

    @Test
    public void gleicheWerte() {
        BinTree h = k(7, blatt(7), blatt(7));
        assertTrue(Baumhueter.isMaxHeap(h), "gleiche Werte sind erlaubt");
    }

    @Test
    public void einzelnesBlatt() {
        assertTrue(Baumhueter.isMaxHeap(blatt(5)), "ein einzelner Knoten ist ein Max-Heap");
    }

    @Test
    public void einKindZuGross() {
        BinTree h = k(9, blatt(12), leer());
        assertFalse(Baumhueter.isMaxHeap(h), "einziges Kind 12 ist groesser als 9");
    }
}
```

Testklasse HuffmanTest (8 Tests zu decodiere)

```
public class HuffmanTest {
```

```
    public HuffmanTest() {
    }

    // Hilfsmethode: innerer Knoten (Inhalt egal) mit den Teilbaeumen l und r
    private BinTree k(Object v, BinTree l, BinTree r) {
        BinTree b = new BinTree(v);
        b.setLeft(l);
        b.setRight(r);
        return b;
    }

    private BinTree z(String s) {
        return new BinTree(s);
    }

    // Codebaum aus Abbildung B.3
    private BinTree codebaum() {
        return k("*", k("*", z("I"), z("E")), k("*", z("S"), k("*", z("R"), z("G"))));
    }

    @Test
    public void beispiel() {
        assertEquals("REISIG", Baumhueter.decodiere(codebaum(), "11001001000111"), "Beispiel aus
Aufgabe b)");
    }

    @Test
    public void einZeichen() {
        assertEquals("I", Baumhueter.decodiere(codebaum(), "00"), "ein Zeichen");
    }

    @Test
    public void leereBitfolge() {
        assertEquals("", Baumhueter.decodiere(codebaum(), ""), "leere Bitfolge");
    }

    @Test
    public void alleZeichen() {
        assertEquals("IESRG", Baumhueter.decodiere(codebaum(), "000110110111"), "alle fuenf Zeichen");
    }

    @Test
    public void langerCodeAmEnde() {
        assertEquals("GRG", Baumhueter.decodiere(codebaum(), "111110111"), "Codewoerter der Laenge 3");
    }

    @Test
    public void huffmanBaumAusA() {
        BinTree t = k(11, k(4, z("E"), k(2, z("D"), z("L"))), k(7, k(3, z("T"), z("A")), z("N")));
        assertEquals("TANNENNADEL", Baumhueter.decodiere(t, "10010111110011111010100011"), "Baum aus
Aufgabe a)");
    }

    @Test
    public void baumUnveraendert() {
        BinTree t = codebaum();
        Baumhueter.decodiere(t, "11001001000111");
        assertEquals("I", (String) t.getLeft().getLeft().getItem(), "Baum unveraendert (links)");
        assertEquals("G", (String) t.getRight().getRight().getRight().getItem(), "Baum unveraendert
(rechts)");
```

```
    }

    @Test
    public void zweiAufrufe() {
        BinTree t = codebaum();
        assertEquals("SIE", Baumhueter.decodiere(t, "100001"), "erster Aufruf");
        assertEquals("SIE", Baumhueter.decodiere(t, "100001"), "zweiter Aufruf liefert dasselbe");
    }
}
```

Kontrolle der Testklassen mit Musterlösung, Alternativen und typischen Fehlvarianten (Mutanten)

Variante von isMaxHeap	HeapTest grün
Musterlösung (Struktogramm zeilengetreu: verschachtelte Abfragen, Rekursion mit und)	8/8
Alternative: Basisfall Blatt, Wahrheitswert ok mit Kurzschluss	8/8
Alternative: Hilfsmethode mit Obergrenze (Wert des Elternknotens)	8/8
keine Rekursion: nur Wurzel und Kinder geprüft	5/8
nur der linke Teilbaum wird rekursiv geprüft	7/8
≥ statt > (gleiche Werte gelten als Verstoß)	7/8
Leer-Abfrage der Kinder fehlt (getItem auf leerem Baum)	3/8
rechtes Kind wird nicht verglichen	5/8
oder statt und bei der Rekursion	5/8
Basisfall Blatt statt leer (Aufruf mit leerem Teilbaum)	7/8
leerer Rumpf (Gerüst, return false;)	4/8

Variante von decodiere	HuffmanTest grün
Musterlösung (Zählschleife, Rücksprung zur Wurzel nach jedem Blatt)	8/8
Alternative: rekursiv mit Index des nächsten Bits	8/8
Alternative: zwei geschachtelte Schleifen, substring/equals	8/8
kein Rücksprung zur Wurzel nach einem Blatt	2/8
Blatt-Prüfung vor dem Schritt (ein Bit zu spät)	2/8
0 und 1 vertauscht	2/8
letztes Bit vergessen (i < bits.length() − 1)	2/8
Vergleich mit der Zahl 0 statt mit dem Zeichen '0'	2/8
Zeichen vorne angefügt (Reihenfolge umgekehrt)	4/8
Baum selbst weiterbewegt (baum = baum.getLeft() …)	2/8
leerer Rumpf (Gerüst, return "";)	2/8

Alternativen zu A d) in Java (für die Bewertung von Struktogrammen mit anderem Aufbau):

Musterlösung (Struktogramm aus Abbildung EH.A3 zeilengetreu):

```
public static boolean isMaxHeap(BinTree b) {
    if (b.isEmpty()) {
        return true;
    }
    int w = (Integer) b.getItem();
    BinTree links = b.getLeft();
    BinTree rechts = b.getRight();
    if (!links.isEmpty()) {
        if ((Integer) links.getItem() > w) {
            return false;
        }
    }
    if (!rechts.isEmpty()) {
        if ((Integer) rechts.getItem() > w) {
            return false;
        }
    }
    return isMaxHeap(links) && isMaxHeap(rechts);
}
```

Alternative (1): Basisfall Blatt

```
public static boolean isMaxHeap(BinTree b) {
    if (b.isLeaf()) {
        return true;
    }
    int w = (Integer) b.getItem();
    boolean ok = true;
    if (!b.getLeft().isEmpty()) {
        ok = (Integer) b.getLeft().getItem() <= w && isMaxHeap(b.getLeft());
    }
    if (ok && !b.getRight().isEmpty()) {
        ok = (Integer) b.getRight().getItem() <= w && isMaxHeap(b.getRight());
    }
    return ok;
}
```

Alternative (2): Hilfsoperation mit Obergrenze

```
public static boolean isMaxHeap(BinTree b) {
    return pruefe(b, (Integer) b.getItem());
}

// jeder Wert darf hoechstens so gross sein wie der Wert des Elternknotens
public static boolean pruefe(BinTree b, int grenze) {
    if (b.isEmpty()) {
        return true;
    }
    int w = (Integer) b.getItem();
    if (w > grenze) {
        return false;
    }
    return pruefe(b.getLeft(), w) && pruefe(b.getRight(), w);
}
```